

(1) 云计算的发展与应用

一、云计算的概念与历史

定义

NIST: 按使用量付费的模式, 提供按需网络访问可配置计算资源池(网络、服务器等), 部署快捷, 管理成本低。

维基百科: 基于互联网的计算方式, 共享资源按需求提供, 类似电力网的规模经济。

核心特点: 用户自主按需申请资源, 资源池化管理。

发展历程

大型机时代: 单机模式, 硬件架构庞大(如 IBM 大型机), 扩展性强。

PC 时代: 计算资源分散, 以个人设备为主。

云计算时代: 整合并行计算(多 CPU 协同)、分布式计算(多设备协作, 如 Hadoop)、网格计算(按地理区域整合资源), 面向全球提供服务。

二、云计算的商业模式与部署模式

商业模式

IaaS (基础设施即服务): 提供云主机、存储等底层资源, 如阿里云主机。

PaaS (平台即服务): 提供数据库、中间件等开发平台, 如大数据服务。

SaaS (软件即服务): 直接提供应用服务, 如 CRM、OA 系统。

类比案例: 景区烧烤场景中, IaaS 是提供场地让用户自建烧烤台,

PaaS 是提供搭建好的烧烤台，SaaS 是提供烤好的食物。

部署模式

私有云：企业自建，安全性高（如政务云），但建设成本高。

公有云：第三方提供，按需付费（如 AWS、阿里云），成本低但存在数据泄露风险，适合中小型企业。

混合云：私有云与公有云结合，核心业务放私有云，非核心业务放公有云（如电商双 11 扩展 Web 服务），降低成本且保障高可用性。

三、虚拟化技术

虚拟机 (VM)

通过 Hypervisor 层模拟硬件，运行完整 Guest OS，镜像体积大 (GB 级)，启动慢（数十秒到几分钟），性能损耗明显。

容器（如 Docker）

轻量级虚拟化，共享宿主机 OS 内核，镜像小 (MB 级)，启动秒级，性能接近物理机，属于“进程级隔离”。

对比：容器与虚拟机的核心区别在于隔离级别（进程级 vs. 操作系统级）、存储大小、部署速度等（见下表）。

对比项	Docker 容器	虚拟机 (VM)
操作系统	与宿主机共享 OS	运行独立虚拟机 OS
存储大小	镜像小 (MB 级)	镜像庞大 (GB 级)

对比项	Docker 容器	虚拟机 (VM)
运行性能	几乎无损耗	有 CPU、内存额外消耗
部署速度	秒级	10 秒以上

3.编排工具

K8S (Kubernetes)：自动化管理容器的部署、扩展和回收，如监控容器状态、负载均衡，被称为“容器的保姆”。

四、应用案例

摩拜单车：基于微软 Azure 实现数据存储、路线推荐、用户账号云端管理。

地震数据处理：阿里云将 30TB / 年的地震数据计算从单机 90 个月缩短至 48 小时，效率提升 1000 倍。

云游戏：通过云服务器流量分发和 CDN 加速，解决下载慢、服务器卡顿问题，支持低配置设备运行大型游戏。

智能音箱（如天猫精灵）：依赖云端 AI 和大数据处理用户指令，本地仅作为终端，实现语音交互、生活服务等功能。

五、市场格局

中国公有云市场份额（2023-2024）

IaaS: 阿里云 (32%)、华为云 (21%)、天翼云 (13%)、中国移动云 (11%)、AWS (8%)。

趋势：公有云市场规模持续增长，2025 年预计达 6500 亿元，混合云和多云成为主流。

全球数据与市场

全球数据总量快速增长，2020 年达 60ZB，80% 数据为近 2 年产生，推动云计算需求上升。

六、未来趋势

技术融合：

云计算 + AI：云计算为 AI 提供算力支持（如模型训练），AI 反哺云计算（如智能资源调度、智能客服）。

大数据 + 云计算：协同处理海量数据，提升分析速度。

安全与合规

全球网络安全投资 2025 年预计突破 2000 亿美元，各国加强云计算监管，安全合规要求趋严。

行业与技术发展

行业云：医疗云、政务云等专属云兴起，SaaS 可能成为主要交付方式。

边缘计算：降低延迟，支持实时场景（如自动驾驶、工业物联网）。

七、总结

云计算以“按需服务、资源池化”为核心，覆盖 IaaS/PaaS/SaaS 三层模式，结合虚拟化、分布式计算等技术，已渗透到生活各领域。未来将与 AI、大数据深度融合，在持续增长的同时，面临安全与合规挑战，行业云与边缘计算将成为重要发展方向。

(2) 云计算的原理与应用

第 2 课：Google 云计算原理与应用（一）

1. Google 文件系统（GFS）基础

核心定位：为 Google 海量服务（如搜索、Gmail）提供底层分布式存储，基于廉价商用硬件构建。

系统架构：

三类节点：

Client：以库文件形式提供接口，负责与应用程序交互，不缓存数据（流式读写场景下缓存收益低）。

Master：唯一管理节点，存储元数据（文件命名空间、Chunk 映射表、副本位置），内存维护元数据并压缩优化。

Chunk Server：存储数据块（默认 64MB/Chunk，分 64KB Block），以 Linux 文件形式存储，数量决定 GFS 规模。

数据流程：控制流（Client→Master）与数据流（Client→Chunk Server）分离，支持 I/O 并行。

容错机制：

Chunk 副本：每个 Chunk 默认 3 个副本，分布在不同节点，丢失时 Master 自动复制。

Master 容错：元数据变更日志记录在本地磁盘并远程备份，启动时通过 Chunk Server 轮询恢复位置信息。

2. 分布式数据处理（MapReduce）基础

产生背景：Jeffrey Dean 设计，解决海量数据并行处理问题，封

装容错、负载均衡等细节。

编程模型：

Map 阶段：输入 (in_key, in_value)，输出中间键值对，如词频统计中 Map 将单词转为 $\langle \text{word}, 1 \rangle$ 。

Reduce 阶段：输入 (key, [value 列表])，归并输出最终结果，如累加单词计数。

处理流程：

输入文件分 M 块，启动 Map Worker 处理。

中间结果分区为 R 份，Reduce Worker 远程读取并合并。

Master 通过 checkpoint 机制容错，Worker 失效时任务重调度。

第 3 课：Google 云计算原理与应用（二）

1. 分布式锁服务（Chubby）与 Paxos 算法

Chubby 设计目标：

提供粗粒度建议性锁服务（文件即锁），支持高可用、扩展性，存储元数据（如 GFS Master 地址）。

建议性锁：进程需主动检查锁存在性，不强制阻止访问，减少系统开销。

Paxos 算法核心：

三类节点：Proposers（提决议）、Acceptors（批决议）、Learners（获决议）。

决议流程：

准备阶段：Proposer 发编号 n 的提案至多数 Acceptors，获取最大已批编号和值。

批准阶段：若多数 Acceptors 接受 n ，决议生效；否则更新 n 重试。

案例：GFS Master 选举中，服务器竞争创建锁文件，成功写入地址的服务器当选。

2. Chubby 系统设计与通信

架构：5 个副本服务器组成 Chubby 单元，通过 Paxos 选主，客户端通过 KeepAlive 协议维持租约（45 秒）。

文件系统特性：

每个文件代表锁，支持共享 / 独占锁，通过 Open/Close/Acquire 等函数操作。

元数据包含实例号、内容生成号等，用于版本控制。

第 4 课：Google 云计算原理与应用（三）

1. 分布式结构化数据表（Bigtable）

数据模型：

多维映射表： $(row_key, column_key, timestamp) \rightarrow value$ ，行按词典序排序，列分组为族（如 Family:Qualifier）。

示例：网页存储中， row_key 为倒排 URL，列族包含“内容”“元数据”，时间戳区分版本。

系统架构：

主服务器：管理子表（Tablet）分配、负载均衡，通过 Chubby

获取子表位置。

子表服务器：存储 SSTable（有序键值对）和预写日志（WAL），子表分裂后自动迁移。

性能优化：

局部性群组：将频繁访问的列族组织为 SSTable，减少 I/O。

压缩与过滤：BMDiff 压缩长串数据，Zippy 快速压缩，布隆过滤器减少磁盘查找。

2. Bigtable 与 Chubby 的集成

Chubby 用于：

选取唯一主服务器，避免脑裂。

存储 Bigtable 模式信息和 ACL（访问控制列表），确保元数据一致性。

第 5 课：Google 云计算原理与应用（四）

1. 分布式存储系统（Megastore）

设计目标：融合关系型数据库（ACID）与 NoSQL（扩展性），支持跨数据中心事务。

数据模型：

实体组（Entity Group）：基本存储单元，类似表，支持 ACID 事务（限于组内）。

示例：用户数据实体组包含 User 表（根表）和 Photo 子表，通过外键关联。

事务与复制：

Paxos 复制：日志通过 Paxos 协议同步到多数副本，确保强一致性。

读写模式：

Current 读：等待所有写操作生效后读取。

Snapshot 读：读取最新完整事务版本。

Inconsistent 读：允许低延迟但可能过期的数据，用于非关键场景。

2. 性能与应用

支持 99.999% 可用性，平均写入延迟 100-400ms，Google 内部超 100 个产品使用（如 App Engine）。

优化措施：

快速读：选择最新副本直接读取，减少网络开销。

快速写：跳过准备阶段，直接提交已批准的日志。

第 6 课：Google 云计算原理与应用（五）

1. 大规模分布式系统监控（Dapper）

设计目标：低开销、透明监控跨服务请求链路，支持故障定位。

核心概念：

监控树（Trace Tree）：由区间（Span）组成，每个 Span 记录 RPC 请求的客户端和服务端注释（如时间、操作）。

关键技术：

二次抽样：首次抽样率 1/1024，写入 Bigtable 前按哈希值二次过滤，减少存储。

数据重组：通过重复深度（r）和定义深度（d）标记嵌套结构，有限状态机（FSM）重组列数据。

工具与应用：

DAPI：支持按监控 ID、批量或索引访问数据。

用户界面：可视化请求延迟分布，定位长尾问题（如网络瓶颈）。

2. 海量数据交互式分析（Dremel）

产生背景：解决 MapReduce 批处理效率低问题，支持秒级查询 TB 级数据。

数据模型与存储：

嵌套列存储：支持原子类型和记录类型（如 Document 包含 Links、Name 嵌套组）。

无损表示：每个列块存储 r（重复深度）和 d（定义深度），如 Name.Language.Code 的 r=0 表示新记录，r=1 表示 Name 重复。

查询与性能：

SQL 支持：嵌套子查询、记录内聚合，如统计 Name.Language.Code 出现次数。

执行架构：多层服务树分发查询，并行扫描数据分片，性能比 MapReduce 提升 1-2 个数量级。

第 7 课：Google 云计算原理与应用（五）（续）

1. 内存大数据分析（PowerDrill）

设计目标：处理 PB 级数据，通过内存缓存和压缩优化查询效率。

核心技术：

双层字典结构：

全局字典：字符串→唯一 ID，减少存储。

块字典：块 ID→全局 ID，加速查询。

数据优化：

分块：按领域专家指定域（如时间、用户 ID）分区，过滤无关块。

编码与压缩：Hyperloglog 估计基数，Zippy 压缩比 GFS 高 22.2%，冷热数据分离（LRU 策略）。

性能指标：

92.41% 数据可跳过，70% 查询无需磁盘访问，平均延迟 25 秒。

2. Google 应用引擎（App Engine）

架构与服务：

提供 PaaS 平台，支持 Java/C++ 开发，自动扩展负载，集成 Memcache、Data Store 等服务。

沙盒限制：

仅允许通过 API 访问网络，禁止写入文件系统，请求处理需在秒级内完成。

应用场景：快速部署 Web 应用，免费域名为 appspot.com，支持自定义域名。

核心技术总结

存储与计算融合：GFS 提供海量存储，MapReduce 实现并行计算，Bigtable/Megastore 处理结构化数据。

分布式一致性：Paxos 算法贯穿 Chubby、Megastore 等组件，确保跨节点数据一致。

性能优化核心：列存储（Dremel）、内存缓存（PowerDrill）、数据分片与并行处理（MapReduce）。

监控与容错：Dapper 追踪全链路请求，各组件通过副本、日志重放等机制保障高可用。

这些技术共同构成 Google 云计算的底层基石，其设计思想（如分布式协作、软件容错、成本优化）为行业提供了经典范式。

(3) AMAZON 云计算 AWS

一、基础存储架构 Dynamo

1. 架构与设计目标

定位：Amazon 构建的分布式 Key-Value 存储系统，支持高可用、容错性和可扩展性，采用去中心化架构，无中心节点。

核心特点：

仅支持键值对存储，不解析数据内容，可存储任意类型数据。

基于分布式哈希表（DHT），通过改进的一致性哈希算法实现数据均衡分布。

2. 关键技术

一致性哈希算法改进：

引入虚拟节点：将物理节点映射为多个虚拟节点，均匀分布在哈希环上，解决节点分布不均问题。

数据分区：将哈希环划分为 Q 个分区，每个虚拟节点负责 Q/S 个分区（ S 为虚拟节点数），提升数据均衡性。

数据备份与容错：

副本数 N （默认 3），存储在哈希环后继节点；采用 Quorum 机制，写操作需 W 个节点确认，读操作需 R 个节点响应（ $W+R>N$ 确保一致性）。

临时故障处理：通过 Hinted Handoff 机制，临时存储数据到其他节点，故障恢复后回传。

永久故障处理：利用 Merkle 哈希树验证数据完整性，检测副本

不一致。

冲突解决：使用向量时钟记录数据版本，解决并发更新冲突，支持最终一致性。

成员管理：基于 Gossip 协议，节点定期交换状态信息，检测故障并更新成员列表，新节点通过“种子节点”接入。

3. 应用场景

存储高频读写的结构化数据，如用户会话、商品信息等。

作为底层存储支持 AWS 其他服务（如 S3、DynamoDB）。

二、弹性计算云 EC2

1. 基本架构

核心组件：

Amazon 机器映像（AMI）：包含操作系统、应用程序的模板，支持公共、私有、共享和付费获取方式，分为 EBS 支持和实例存储支持两类。

实例（Instance）：基于 AMI 启动的虚拟服务器，支持计算、内存、GPU 等多种类型，可按需调整类型，临时存储随实例终止丢失。

弹性块存储（EBS）：持久化存储卷，支持快照备份到 S3，适用于数据库主存储。

网络组件：私有 IP、公共 IP、弹性 IP（静态绑定账户），通过 NAT 转换实现地址映射。

区域与可用区：

地理区域（Region）按地理位置划分（如美东、亚太），可用区

(Zone) 为区域内独立数据中心，实例分布在不同可用区提升容错性。

2. 关键技术

弹性负载均衡：自动分发流量，识别故障实例并路由流量到健康实例。

监控与自动缩放：

CloudWatch：监控 CPU、磁盘、网络等指标。

自动缩放：根据预设条件（如负载）自动调整实例数量，适应流量波动。

通信机制：

弹性 IP 地址绑定账户，实例故障时重新映射到新实例，避免 DNS 更新延迟。

3. 安全与容错

安全组：自定义规则控制实例网络流量，默认拒绝非授权访问。

SSH 密钥对：通过公钥加密登录，私钥本地保存，公钥存储在 EC2 实例元数据中。

容错设计：实例故障时，弹性 IP 重新映射，EBS 快照确保数据持久化。

三、简单存储服务 S3

1. 基本概念与操作

存储结构：

桶 (Bucket)：存储容器，全局唯一，无嵌套，数量受限但对象数无限制。

对象 (Object)：基本存储单元，由数据、元数据（如 Last-Modified、Content-Type）和键 (Key) 组成，键创建后不可修改。

操作类型：

桶操作：创建、删除、列出对象；**对象操作：**上传 (Put)、下载 (Get)、删除 (Delete)、查询元数据 (Head)。

2. 数据一致性模型

最终一致性：写操作后立即读取可能返回旧数据，适用于非实时场景，典型场景如下：

写入新对象立即读取可能返回“键不存在”。

删除对象后立即读取可能仍返回旧数据。

3. 安全措施

身份认证：

Access Key ID + Secret Access Key：用户注册获取，用于生成 HMAC-SHA1 签名，验证请求合法性。

访问控制列表 (ACL)：

支持五种权限 (READ、WRITE、READ_ACP、WRITE_ACP、FULL_CONTROL)，桶和对象 ACL 独立，不具继承性。

授权用户类型：所有者、个人用户 (邮箱 / 用户 ID)、组 (AWS 用户组 / 所有用户组)。

四、非关系型数据库服务 (SimpleDB & DynamoDB)

1. SimpleDB

数据模型：

域 (Domain): 数据容器, 域名唯一, UTF-8 字符串存储, 查询限于单域。

条目 (Item): 记录, 由属性 (Attribute) 组成, 无需预定义模式, 属性值 $\leq 1\text{KB}$ 。

属性与值: 属性描述条目特征, 值支持多值 (如商品颜色属性存多个值)。

限制与解决方案:

数据量有限, 大文件存储在 S3, SimpleDB 仅存储文件指针。

2. DynamoDB

改进点:

取消表大小限制, 自动分片到多服务器, 支持 TB 级数据。

支持强一致性和弱一致性选择, 默认最终一致性。

硬件优化: 使用 SSD, 根据读写流量预设分配硬盘数量。

3. 两者对比

SimpleDB: 适合小规模复杂查询, 自动索引所有属性, 查询功能强。

DynamoDB: 适合大规模负载, 支持自动扩展, 灵活性和性能更优。

五、关系数据库服务 RDS

1. 基本原理

架构:

基于 MySQL 集群, 采用 Share-Nothing 架构, 服务器独立不

共享资源，通过表单划分（Sharding）拆分大表到多服务器。

主从备份与读副本：主库写，从库读，主库故障时从库自动接管，支持版本升级时先升级从库再切换。

2. 使用方式

DB Instance：MySQL 服务器实例，存储 5GB~1TB，通过 Web API 创建，使用标准 MySQL 协议操作。

工具支持：命令行工具（Java）和兼容的 MySQL 客户端（如 Navicat）。

六、简单队列服务 SQS

1. 基本模型

组件组成：

队列（Queue）：消息容器，类似 S3 桶，名称唯一，支持先进先出（FIFO）。

消息（Message）：文本数据，大小受限（通常 $\leq 256\text{KB}$ ），数量无限制，包含消息 ID、体、接收句柄、MD5 摘要。

2. 消息机制

取样与可见性：

加权随机取样：消息冗余存储在多服务器，查询时随机选取部分服务器返回副本，解决消息查询不准确问题。

可见性超时：消息接收后设为不可见，超时未删除则重新可见并重传，用户可扩展或终止计时。

生命周期：消息未删除时保留 4 天，删除后彻底移除。

七、内容推送服务 CloudFront

1. CDN 基础

传统网络问题：

服务器访问量有限、地域差异导致延迟、不同网络服务商互访慢。

CDN 解决方案：

边缘节点靠近用户，智能 DNS 负载均衡返回最近边缘节点 IP，减少路由跳转。

2. CloudFront 优势

收费模式：按使用量付费，适合中小企业。

集成与安全：

与 S3 集成，部署简单；仅支持 HTTPS 访问，提升安全性。

核心概念：

对象、源服务器、分发：对象为分发文件，源服务器存储文件，分发建立 CloudFront 与源服务器通道。

边缘节点位置、有效期：节点全球分布，文件副本在边缘节点存储时间可设置。

八、核心技术总结

分布式存储与计算：Dynamo 的一致性哈希与虚拟节点、EC2 的弹性扩展、S3 的对象存储构成 AWS 底层基础。

数据一致性：S3、DynamoDB 采用最终一致性，RDS 通过主从备份保证强一致性，适应不同场景需求。

服务解耦与通信：SQS 的队列机制解耦组件通信，CloudFront

的 CDN 加速内容分发，提升用户体验。

安全与容错：各服务通过 ACL、密钥对、区域 / 可用区设计，确保数据安全性与服务高可用。

这些服务相互协同，形成 AWS 完整的云计算生态，为用户提供从存储、计算到数据处理的一站式解决方案。

(4) 虚拟化技术

一、虚拟化技术概述

定义与核心思想

虚拟化技术通过软件或固件构建虚拟化层，将物理资源（如服务器、存储、网络）抽象为虚拟资源，实现多用户共享与动态分配。其核心是利用虚拟化层映射物理资源，使多个虚拟机可在同一物理设备上独立运行。

在云计算中的作用

构建虚拟数据中心，整合物理资源，提高利用率（如服务器利用率从 10%-15% 提升至 60%-80%）。

支持资源动态调度、自动化部署，降低运维成本。

提供安全隔离与可靠性机制，满足不同业务需求。

二、服务器虚拟化

层次与架构

寄居虚拟化：VMM 安装在宿主操作系统上（如 VMware Workstation），依赖宿主管理资源，系统损耗大。

裸机虚拟化（Hypervisor）：直接运行于硬件之上（如 VMware ESXi、Xen），通过 Hypervisor 管理硬件，性能更优。

底层实现技术

CPU 虚拟化：通过模拟执行或硬件辅助（如 Intel VT-x、AMD SVM）实现虚拟 CPU 调度，解决 x86 架构特权指令捕获问题。

内存虚拟化：利用页表映射（如 EPT/NPT）管理虚拟地址与物理

地址转换，支持写时复制（Copy on Write）共享内存。

I/O 设备虚拟化：通过软件模拟（如 QEMU）或硬件辅助（如 Intel VT-D）实现设备分配，优化 I/O 性能。

虚拟机迁移

动态迁移（Live Migration）：在不中断服务的前提下，将虚拟机从一台物理机迁移至另一台，步骤包括预复制、停机复制、提交等，关键是内存迁移（分 Push、Stop-and-Copy、Pull 阶段）。

迁移内容：包括内存状态、网络配置（ARP 重定向保持 IP/MAC 绑定）、存储（通过 NAS 共享而非复制）。

隔离技术

内存隔离：通过 MMU 和页表映射确保虚拟机内存独立，如 Xen 的地址空间隔离。

网络隔离：利用虚拟交换机（如 vSwitch）、VLAN、ACL 实现流量控制与安全隔离。

三、存储虚拟化

核心概念

将分散的异构存储设备抽象为统一的虚拟存储池，通过虚拟化引擎管理逻辑卷与物理存储的映射，屏蔽底层设备差异。

实现方式

基于主机：通过逻辑卷管理（LVM）在服务器端实现，性价比高但扩展性差（如 Linux LVM）。

基于存储设备：在存储阵列控制器中实现（如 Dell EMC 存储虚

拟化)，与主机无关，但异构支持差。

基于网络：通过网络设备（如 FC 交换机、路由器）实现，支持动态多路径，但故障影响范围大。

案例：VMware VMFS

支持多主机并发访问同一存储，通过磁盘锁定、故障一致性机制确保数据安全，裸机映射（RDM）允许虚拟机直接访问物理存储 LUN。

四、网络虚拟化

核心层与接入层虚拟化

核心层：虚拟化核心网络设备，提供万兆接入与虚拟机箱技术，简化管理（如 Cisco Nexus 虚拟机箱）。

接入层：引入无损以太网技术（如 PFC、ETS），减少拥塞，支持虚拟机迁移大流量传输。

虚拟机网络虚拟化

虚拟交换机（vSwitch）：在主机内虚拟交换机功能，支持 VLAN、QoS、负载均衡（如 VMware vSphere 的 vSwitch）。

虚拟网卡（vNIC）：在物理网卡上虚拟多个逻辑网卡，每个 vNIC 独立配置 MAC/IP，支持流量调度。

案例：VMware vNetwork

分布式交换机（dvSwitch）统一管理跨主机网络配置，虚拟机迁移时保持网络策略一致；端口组定义网络策略，支持 VLAN、网卡绑定。

五、桌面虚拟化

技术定位

将桌面系统的运行环境与硬件解耦，用户通过协议（如 RDP、ICA）

(5) 微软云计算 Windows Azure

第 13 课：微软云计算 Windows Azure（一）

4.1 微软云计算平台概述

发展历程

2008 年推出 Windows Azure Service Platform，2014 年更名为 Microsoft Azure，支持多语言开发（如 PHP、Python、Java 等），不再局限于 Windows 生态。

2014 年 3 月，世纪互联开始运营中国大陆地区的 Windows Azure 公有云服务。

平台定位与组成

PaaS 模式：面向开发者，提供计算、存储、数据库等服务，核心组件包括：

Windows Azure：云操作系统，负责资源调度与管理。

SQL Azure：云端关系数据库服务。

Windows Azure AppFabric：提供分布式架构服务（如服务总线、访问控制）。

Windows Azure Marketplace：云服务与数据交易市场。

与传统部署方式的对比

传统方式：自建服务器或租用虚拟主机，成本高且控制权受限。

微软云优势：按需付费、弹性扩展，用户无需管理底层硬件，专注应用开发。

4.2 微软云操作系统 Windows Azure

体系架构核心组件

计算服务：支持 Web Role、Worker Role、VM Role 三类实例：

Web Role：基于 IIS 部署 Web 应用，自动绑定 HTTP/HTTPS 端口。

Worker Role：运行后台任务，如数据处理、消息队列消费。

VM Role：支持用户自定义 Hyper-V 虚拟机镜像，提供 IaaS 能力。

存储服务：包含 Blob、Table、Queue、File 四种数据结构：

Blob：存储非结构化数据（如图片、视频），支持海量存储。

Table：非关系型结构化数据存储，类似 NoSQL，查询语言非 SQL。

Queue：应用组件间的消息通信队列，支持异步处理。

File：通过 SMB 协议共享文件，本地应用可直接访问。

Fabric 控制器：分布式管理组件，负责虚拟机部署、监控与故障转移，通过 Fabric 代理与节点通信。

Windows Azure Connect：基于 IPsec 建立本地与云端的安全连接，类似 VPN，支持访问本地资源（如共享文件夹、数据库）。

Windows Azure CDN：内容分发网络，缓存 Blob 副本至全球边缘节点，提升访问速度。

存储服务底层架构（WAS）

全局命名空间：

通 过 URI 访 问 数 据

([http\(s\)://AccountName.<service>.core.windows.net/PartitionName/ObjectName](http(s)://AccountName.<service>.core.windows.net/PartitionName/ObjectName))，支持无限扩展。

双复制引擎：

域内复制（同步）：文件流层将数据同步复制 3 份至同一数据中心的不同节点，确保硬件故障时的数据可用性。

域间复制（异步）：分区层跨数据中心异步复制，防止地域灾难，不影响用户请求延迟。

文件流层：分布式文件系统，支持追加写（数据块不可修改），通过 Paxos 协议保证副本一致性，区块（Extent）作为存储单元，每个区块复制 3 份至不同节点。

分区层：管理数据对象（Blob/Table/Queue）的事务与负载均衡，将对象表拆分为分区段（RangePartition），动态分配至分区服务器，支持分区拆分与合并以平衡负载。

第 14 课：微软云计算 Windows Azure（二）

4.3 微软云关系数据库 SQL Azure

服务架构与组件

核心服务：

SQL Azure 数据库：云端关系型数据库，支持 TDS（表格格式数据流）和 T-SQL，兼容 SQL Server 数据模型，自动管理备份、高可用与负载均衡。

SQL Azure 报表服务：基于 SSRS（SQL Server Reporting Services）的云版本，支持创建、发布报表至门户或嵌入 ISV 应用，不支持调度与订阅功能。

SQL Azure 数据同步：通过“轮辐式（hub-and-spoke）”模型，实

现云端数据库与本地 SQL Server、不同云端数据库间的数据同步，支持增量更新以提升效率。

与 SQL Server 的对比

管理差异：

SQL Azure 自动处理物理资源（如存储、备份），用户无法指定索引位置或手动备份，而 SQL Server 需用户自行管理硬件与备份。

功能限制：

不支持部分依赖物理路径的 T-SQL 参数（如文件组配置），不支持 SQL Server 分析器、数据库引擎优化顾问等工具。

一致性模型：默认最终一致性，通过 Quorum 机制 ($W+R>N$) 保证读写一致性（W 为写确认节点数，R 为读响应节点数）。

应用场景

适合中小型企业的云原生应用，无需部署本地数据库；支持与本地系统混合部署，通过数据同步实现云端与本地数据互通。

4.4 Windows Azure AppFabric

服务总线（Service Bus）

功能：解决跨网络服务通信问题，支持 WCF 服务注册与发现，通过中继服务穿透防火墙与 NAT，无需开放入站端口。

通信流程：

WCF 服务注册终端至服务总线，获取 URI 地址。

客户端通过 URI 调用服务，消息经服务总线中继转发至本地服务端，无需直接连接。

附加特性：

消息缓冲：队列存储消息，支持异步处理，确保服务不可用时消息不丢失。

负载均衡：监听服务随机分发请求至多个 WCF 实例，提升可用性。

访问控制（Access Control）

身份认证流程：

用户访问应用时重定向至身份提供者（如 Windows Live ID、Facebook）。

身份提供者验证用户后返回 IDP Token。

访问控制服务验证 IDP Token，根据规则生成 AC Token（包含用户声明）。

应用验证 AC Token，授权用户访问。

优势：支持多身份源集成，简化应用的认证逻辑，无需自建身份系统。

高速缓存服务

分布式缓存：为 Windows Azure 应用提供内存缓存，减少数据库访问频次，支持本地缓存与共享缓存自动同步，提升高频访问数据的读取速度。

核心技术总结

Windows Azure 的分层设计：

从底层存储（WAS）到计算服务（三类 Role），再到上层数据库（SQL Azure）与中间件（AppFabric），形成完整 PaaS 生态。

数据一致性策略：

存储服务通过域内 / 域间复制保证高可用，SQL Azure 结合 Quorum 机制与自动备份，平衡性能与可靠性。

混合云能力：

Windows Azure Connect 与数据同步服务支持本地与云端数据互通，适应企业数字化转型中的混合部署需求。

开发者友好性：

兼容多语言开发，提供可视化工具（如 Visual Studio）与标准化接口（如 WCF、T-SQL），降低迁移成本。